

WARSTWA KONTROLI NA WYJŚCIU DO AI

AegIs

Lokalna bramka między pracownikiem (albo agentem) a modelem. Prompt nie wychodzi z firmy, dopóki nie przejdzie kontroli.

Co jest w tej prezentacji

- 01

Problem: AI w pracy i dane, które już wyciekają
- 02

Dlaczego DLP, proxy i szkolenia nie wystarczą
- 03

Czym jest Aegls i gdzie siada w ruchu
- 04

Jak zapada decyzja: allow / redact / block
- 05

Kategorie, pewność, role i ludzie
- 06

Incydent: kto, co, kiedy, dlaczego
- 07

Panel, zmiana polityki bez restartu
- 08

Uruchomienie i granice produktu

Ludzie wklejają pracę do chata.

38%

w pracy, do chata

Firmy wpuszczają ChatGPT, Copilota i agentów do codziennej roboty. Pracownik nie „atakuję systemu”. On się spieszy: prosi o recenzję commita, podsumowanie umowy, draft maila do klienta.

Do pola wiadomości ląduje to, nad czym właśnie siedzi — kod, tabela z HR, zrzut z CRM, folder z `.env`. Model jest poza firmą. Jak prompt wyjdzie, nie da się go wycofać. Dostawca widzi treść, trenuje, loguje, czasem pokazuje ją w innym wątku.

To nie jest teoretyczne ryzyko RODO. To nawyk. AegIs zakłada, że nawyku nie wyćwiczysz — trzeba go zatrzymać *zanim* pakiet opuści maszynę.

Jeden wklejony folder.

Kuba z frontendu prosi ChatGPT o recenzję commita przed pushem. Model odpowiada: „wklej diff albo wrzuć pliki”. Kuba wrzuca cały katalog, bo nie chce mu się cenzurować.

- .env
- PESEL_Kowalski.txt
- wypłaty_marzec.xlsx

SEKRET

Klucze w .env

Token do bazy, Stripe, Slack. Po wklejeniu jest w historii chata.
Rotacja to już gaszenie pożaru.

OSOBA

Identyfikatory

PESEL, NIP, e-mail, telefon. Dla RODO to udostępnienie danych podmiotowi spoza umowy powierzenia.

FIRMA

Wynagrodzenia

Arkusze z pensjami nie jest „kontekstem do recenzji kodu”. Jest dokumentem kadrowym w obcym modelu.

Dlaczego dotychczasowe narzędzia tego nie łapiają

DLP na mailu i pendrive

Widzi załącznik w Outlooku i kopiowanie na USB. Nie widzi pola wiadomości na chatgpt.com, claude.ai ani okna agenta. Ruch idzie HTTPS-em z przeglądarki użytkownika, często z konta prywatnego.

Proxy bez treści

Zwykły HTTP proxy po HTTPS widzi host (SNI), nie prompt. Żeby czytać ciało, trzeba firmowego CA i MITM — a i tak nie wejdiesz w spięty pinning konsumenckich czatów. To zła droga.

Szkolenie i regulamin

Ludzie znają zakaz. Potem mają deadline. Jeden „wrzucić cały folder” obchodzi politykę, której nikt nie egzekwuje w momencie Enter.

Agenty poza przeglądarką

SDK, cron, LangChain, skrypt w CI wołają API wprost. Nie ma tam wtyczki. Potrzebują bramki na `OPENAI_BASE_URL` albo firmowego proxy — tego samego silnika decyzji.

AegIs jest bramką, nie kolejnym chatbotem.

Siedzi między tym, kto pisze (człowiek albo program), a modelem. Każdy prompt jest sprawdzany lokalnie. Dopiero werdykt decyduje, czy cokolwiek poleci na zewnątrz.

01 AGENT

pracownik / skrypt

Pisze wiadomość w ChatGPT, Claude, albo woła API z kodu.

02 AEGIS

bramka

Na tej maszynie albo w firmowym procesie: finder, klasyfikator, reguły, rola. Nic nie „dosyła do chmury, żeby sprawdzić”.

03 MODEL

dostaje albo nic

Allow — idzie oryginał. Redact — idzie z maskami. Block — model nie dostaje promptu, w panelu ląduje incydent.

ChatGPT, Copilot i agent zostają. Nie każemy ludziom przenosić się do „bezpiecznego chatu firmy”. Zostawiamy im narzędzia, zabieramy wyciek.

Trzy drogi, jeden silnik.

Nieważne, czy ktoś klepie w przeglądarce, czy agent woła API. Werdykt, polityka i dziennik są te same.

PRZEGŁĄDARKA

Wtyczka Chrome

Czyta pole wiadomości na ChatGPT, Claude, Gemini, Copilot, Perplexity, Grok, DeepSeek i innych. Przed Enter / „Wyślij” woła `POST /v1/scan`. Potrafi przerwać wysyłkę i pokazać, co zablokowała.

AGENCY / SDK

Bramka API

Aplikacja ustawia `OPENAI_BASE_URL` na `http://127.0.0.1:8080/net/openai/v1`. To samo dla Anthropic i Ollama. Helios skanuje JSON promptu, potem — jeśli wolno — puszcza upstream.

PROCESY

HTTP proxy :8888

Dla narzędzi z `HTTP_PROXY`. Po HTTP widać ciało. Po HTTPS bez CA — host. Dlatego agenty AI i tak powinny iść BASE_URL-em, nie „magicznym MITM wszystkiego”.

Trzy wyjścia z bramki

ALLOW Puść Brak wrażliwych spanów, albo rola / whitelist mówi, że ta osoba może to wysłać. Model dostaje oryginał. Zapis w audycie zostaje.	REDACT Zredaguj PESEL, e-mail, karta, telefon zamieniają się w maski ([PESEL] , [EMAIL]). Reszta promptu idzie. Pracownik widzi, co ucięto — może poprawić i wysłać jeszcze raz.	BLOCK Zatrzymaj Sekrety, identyfikatory krytyczne, jailbreak. W UI stop, w API 403 . Model nie dostaje nic. W dashboardzie powstaje incydent: kto, co, kiedy, która kontrolka.
---	--	--

Domyślny preset to **balanced**: government_id , financial , credentials i attack idą w block; zdrowie w redact. Admin zmienia to suwakiem, nie deployem.

Jak w ogóle widzimy, że to PESEL

Finder	Wycina kandydatów tymi samymi rodzinami regex co trening Laya (pii), plus PESEL, NIP, REGON, IBAN, klucze API, tokeny, hasła.
Laya · pii	Otwarty klasyfikator decyzji (wagi Apache-2.0). 22 etykiety: imię, telefon, karta, paszport... Dostaje span i zamkniętą listę odpowiedzi, zwraca skalibrowane prawdopodobieństwa. Nie generuje tekstu i nie wysyła treści do zamkniętego API.
Reguły twarde	Poprawny PESEL / NIP, Luhn karty, IBAN i sekrety zostają nawet gdy klasyfikator powie not pii . Checksumy nie są „opinią modelu”.
Sygnatury	Jailbreak, prompt injection, eksfiltracja, nadużycie narzędzi — katalog w stylu OWASP LLM, import z CERT, włączany per kontrolka.
Heurystyka	Bez pobrania wag (~850 MB) bramka i tak wstaje: backend=heuristic . Semantyczne kontrolki pokazują się jako wyłączone, reszta działa. Na jury i na laptopie sędziego to ten sam produkt.

Admin układu, co jest czym — i jak pewnie.

Kategoria to nie sztywna lista z regulaminu. To etykieta z ryzykiem, akcją i progiem pewności. Laya ocenia span; jeśli pewność jest poniżej suwaka, kategoria nie strzela.

KATEGORIA	PRZYKŁADY	DOMYŚLNA AKCJA
government_id	PESEL, NIP, paszport, dowód	block
credentials	klucz API, token, hasło, prywatny klucz	block
financial	karta (Luhn), IBAN	block
attack	jailbreak, injection, eksfiltracja	block
health	historia choroby, skierowania	redact
contact / person_name	e-mail, telefon, imię	zależnie od destynacji i roli

Własne kategorie dodajesz w panelu. Presety `strict` | `balanced` | `permissive` . Nadpisanie per destynacja: inna polityka na localhost, inna na chatgpt.com. `policy.json` przeładowuje się po mtime — bez restartu procesu.

Rola to wyjątek, nie opis stanowiska.

Programista, lekarz i księgowa wyciekają czymś innym. Jedna globalna blokada albo wszystkich wkurzy, albo nic nie złapie. Dlatego rola mówi: *tej osobie wolno wysłać te kategorie*.

ROLA	WYJĄTKI	SENS
Pracownik	brak	Każda wycieczka PII ląduje u managera. To domyślny stan nowej osoby.
HR	imię, kontakt, demografia	Rekrutacja musi móc wkleić imię i telefon kandydata. PESEL nadal nie.
Finanse	financjal	IBAN kontrahenta albo karta testowa nie jest incydentem. Klucz API nadal jest.
Manager	wszystko + override	Widzi dashboard, może nadpisać blokadę, gdy naprawdę trzeba.

Ta sama kategoria może być blokiem dla developera i allow dla księgowej. Role edytujesz w panelu; lista wyjątków to checkboxy kategorii, nie nowy deploy.

Przypisanie działu od następnej wiadomości.

Strona Pracownicy

Każda osoba ma e-mail, zespół i `role_id`. Wtyczka i bramka wiedzą, kto pisze (konto / nagłówek `X-Employee-Id`). Werdykt liczy się wobec *tej* roli, nie wobec „wszystkich w firmie”.

Nieznany nadawca

Ktoś wszedł z konta, którego nie ma na liście. Trafia do kolejki „nieznani” — nie znika w logach i nie dostaje cichych wyjątków managera.

Bez restartu

Zmieniasz Magdzie rolę z Pracownik na HR. Nie przeładowujesz wtyczki, nie restartujesz bramki, nie budujesz obrazu Dockera. Kolejny Enter idzie już nową polityką. Baner w panelu pokazuje nową wersję.

Whitelist

Osobne wyjątki bez alarmu: adres testowy, numer konta firmowego, fraza, która jest fałszywym alarmem. Whitelist nie wyłącza reszty bramki.

Blok bez incydentu to teatr.

Jak AegIs zatrzyma prompt, w dashboardzie powstaje rekord. Nie „jakiś alert”. Konkretnie: da się przeczytać i zareagować.

Co jest w rekordzie

- kto — osoba, e-mail, zespół, rola
- gdzie — chatgpt.com, OpenAI API, agent, proxy
- co — kategorie i etykiety, fragment z maskami
- dlaczego — które kontrolki strzeliły, pewność, akcja
- kiedy — znacznik czasu i opóźnienie bramki

Co z tym robi manager

- kolejka z filtrami, grupowanie powtórek (dedupe 5 min)
- akcje zbiorcze, przypisanie, notatka, runbook
- eksport CSV / JSONL do audytu
- wartości zamaskowane; `reveal` sam wpada do dziennika

To jest ślad pod RODO i pod rozmowę z człowiekiem („wkleiłeś wypłaty”), nie zrzut całego czata. Treść nie idzie do modelu — i nie musi iść do chmury SIEM, żeby w ogóle powstać.

Wszystko, co jury może zmienić, widać od razu.

MONITOR

Postawa, incydenty, audit

Wynik 0–100, karty „dzisiaj”, słupki 24h, top osób i destynacji. Kolejka incydentów. Dziennik append-only. Telemetry: p50/p95, RPM, podział backendu, błędy upstreamu. Pole „sprawdź prompt” woła ten sam `/v1/scan`.

GOVERN

Kontrolki i polityka

Włącz / wyłącz checksumy, klasyfikator, sygnatury, role, whitelist. Suwak progu. Akcja per kategoria i per destynacja. Presety. Historia `policy.json`. Role, kategorie, ludzie — edycja w miejscu.

NA ŻYWO

Następne żądanie, nie restart

Zmiana threshold, wyłączenie kontrolki albo przypisanie roli pokazuje baner „Policy reloaded (version N)”. `GET /health` zwraca nową wersję. Czat demo idzie pełną ścieżką przez `/net/openai`.

DEGRADACJA

Bez Laya też wstaje

Brak wag = heurystyka + baner. Deterministyczne PESEL / NIP / IBAN / Luhn / sekrety / sygnatury zostają. Semantyka jest szara, nie kłamie że „model czuwa”.

Stop na maszynie. W ~28 ms.

Klasyfikacja nie jest round-tripem do dostawcy modelu. Finder i reguły twarde są tanie. Laya, jeśli włączona, liczy się lokalnie. Stąd rząd wielkości: dziesiątki milisekund, nie sekundy „poczekaj, skanujemy w chmurze”.

To ma znaczenie przy Enter w czacie i przy agencie, który i tak już czeka na tokeny. Bramka nie może być drugim modelem.

Jeśli werdykt to block — pakiet umiera tutaj. Nie ma „wyślemy, a potem zrobimy incident response”.

KOLEJNOŚĆ

Jedno żądanie

allow-lista modeli → rate limit → budżet → loop guard → finder + klasyfikator →
sygnatury → wzorce → wyłączone kontrolki → whitelist / rola → akcja per kategoria
→ redakcja → incydent → audit + telemetria.

Czego świadomie nie robimy

Nie łamiemy konsumenckich chatów

Brak MITM na Messengerze, WhatsAppie, pinned TLS Cursor. Tam nie ma `OPENAI_BASE_URL`. Do chatów AI w przeglądarce jest wtyczka, która czyta pole, nie cały TLS sesji.

Nie udajemy, że chmura sklasyfikuje lepiej

Wysyłanie promptu do obcego API „żeby sprawdzić, czy jest wrażliwy” jest tym samym wyciekiem. AegIs ma działać bez tego kroku.

Nie jesteśmy DLP na wszystko

Nie zastępujemy antywirusa, szyfrowania dysku ani DLP na poczcie. Jesteśmy warstwą na *wyjściu do modelu*. Reszta stosu zostaje.

Nie chowamy degradacji

Bez Laya widać heurystykę. Bez klucza upstreamu bramka i tak zablokuje wcześniej — 502 od dostawcy nie maskuje 403 od nas.

Po clone — jedna komenda.

Na PATH: Python 3.10+ i Node 18+. Resztę robi skrypt: venv, zależności, build dashboardu, bramka i proxy.

INSTALACJA

```
python start.py
```

Windows: `py start.py` albo `.\start.ps1`. macOS / Linux: `./start.sh`. Domyślnie heurystyka (szybki start). `--with-laya` ściąga model.

ADRESY

Co wstaje

Dashboard `http://127.0.0.1:8080/app` · demo chat `/demo` · API `/v1/scan` · bramka `/net/openai/v1` · proxy `:8888`.

Wtyczka: Chrome → `chrome://extensions` → tryb deweloperski → Załaduj rozpakowane → katalog `extension/`. Potem ChatGPT / Claude / Gemini — w rogu znacznik, przy PESEL-u Enter nie wychodzi.

AegIs

Lokalna bramka. Prompt nie wychodzi, dopóki nie przejdzie kontroli. Polityka per kategoria i per rola. Incydent mówi kto i co — zanim model zdąży zobaczyć cokolwiek.